Artificial Neural Networks

MASTERING DEEP LEARNING FOUNDATIONS & MATHEMATICAL ARCHITECTURES

1. What is Deep Learning & Deep Learning Fundamentals

Deep Learning is a specialized subfield of Machine Learning based completely on **Artificial Neural Networks (ANNs)**. The term "deep" directly describes the architectural presence of multiple *hidden layers* stacked sequentially between the initial input layer and the ultimate output layer. While traditional machine learning techniques rely heavily on manual feature engineering (where a human expert extracts key descriptors from the raw data), Deep Learning models execute **feature hierarchy learning**—automatically extracting increasingly complex abstractions at every layer of the deep network stack.

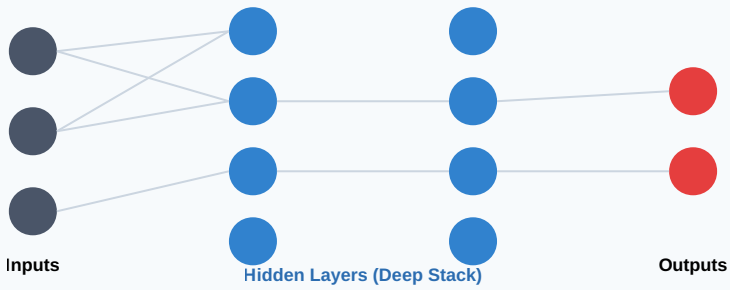

FIGURE 1: FULLY CONNECTED DEEP NEURAL NETWORK ARCHITECTURE

2. How Do Artificial Neurons Work? A Complete Guide

The basic mathematical operational block of an ANN is the **Artificial Neuron** (often called a *Perceptron*). It models biological synapses by acquiring numerical input values, scaling them individually to weight parameters, compiling them via a summation junction, and transforming the net scalar using an analytical operational step function.

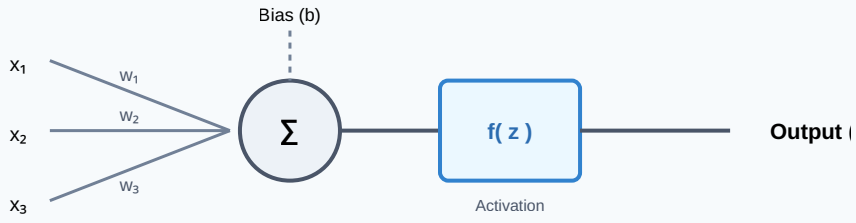

FIGURE 2: MATHEMATICAL SCHEMA OF AN ARTIFICIAL NEURON

The processing inside a neuron follows a two-stage mathematical sequence:

- **Linear Combination (z):** Inputs are multiplied by their respective weights, and a constant scalar shift known as the *bias* (b) is added. The bias gives the neuron the freedom to shift its activation threshold left or right regardless of the input values.

$$z = \sum_{i=1}^{n} w_i x_i + b = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

- **Non-Linear Mapping (y):** The intermediate value z passes through an activation function, $f(z)$, yielding the final layer representation: $y = f(z)$.

3. Activation Functions Explained: ReLU, Sigmoid, Tanh & Threshold

Without activation functions, any multi-layer neural network collapses into a single large linear regression model, completely unable to map complex, non-linear real-world boundaries. These functions inject the essential non-linear transformations required to handle intricate data structures.

Function	Mathematical Form	Range	Primary Use Case & Key Properties
Threshold (Binary Step)	$$f(z) = egin{cases} 1 & ext{if } z \geq 0 \ 0 & ext{if } z < 0 \end{cases}$$	$$\{0, 1\}$$	Early networks (Perceptrons). Limited utility because its zero derivative everywhere prevents gradient descent updates.
Sigmoid	$$f(z) = rac{1}{1 + e^{-z}}$$	$$(0, 1)$$	Binary classification output layers. Prone to the *vanishing gradient problem* under extreme input values.
Tanh (Hyperbolic Tangent)	$$f(z) = rac{e^z - e^{-z}}{e^z + e^{-z}}$$	$$(-1, 1)$$	Hidden layers. Zero-centered outputs mean negative inputs map to strong negative activations, helping center updates.
ReLU (Rectified Linear Unit)	$$f(z) = \max(0, z)$$	$$[0, \infty)$$	Standard default choice for hidden layers. Extremely efficient to compute; avoids vanishing gradients for positive values.

> **The Dying ReLU Phenomenon:** When an input value z falls below zero, the gradient of the standard ReLU function becomes exactly 0. If a neuron's weights shift such that it always receives negative inputs, its weights stop updating entirely. This leaves the neuron permanently inactive—a state known as a "dead neuron." Variants like **Leaky ReLU** address this by introducing a small, non-zero slope (e.g., $f(z) = \max(0.01z, z)$) for negative values.

4. How Do Neural Networks Work? A Step-by-Step Property Valuation

To understand how these concepts map to a real-world scenario, consider a neural network trained for **Property Valuation** (predicting real estate prices based on feature variables):

- **The Inputs (x):** Let $x_1 = ext{Area (sq ft)}$, $x_2 = ext{Number of Bedrooms}$, $x_3 = ext{Distance to Transit (miles)}$, and $x_4 = ext{Crime Rating}$.
- **Hidden Layer Feature Mapping:** When these parameters stream forward through the network, the hidden layer nodes automatically construct higher-level concepts without explicit human definitions:
  - *Hidden Node 1 (Walkability):* Combines x_3 (Distance to Transit) and x_4 (Crime Rating) with heavily weighted parameters to compute a localized safety and convenience score.
  - *Hidden Node 2 (Family Suitability):* Combines x_1 (Area) and x_2 (Bedrooms) to quantify spatial livability.
- **The Output (y):** The final output layer aggregates the outputs of these hidden nodes, applying its own weights to produce a continuous numerical value: the final predicted market valuation.

5. How Do Neural Networks Learn? Cost Functions & Backpropagation

A network initializes with completely random weight variables, leading to highly inaccurate initial predictions. Learning is the iterative process of adjusting those weights to minimize the difference between the model's predictions and the true labels.

The Cost Function (J)

The cost function measures the model's performance by calculating the total error across the training dataset. For regression problems like property valuation, we often use **Mean Squared Error (MSE)**:

$$J(W, b) = rac{1}{2n} \sum_{i=1}^{n} (\hat{y}^{(i)} - y^{(i)})^2$$

Where $\hat{y}$ is the model's prediction, y is the true ground-truth value, and n is the total number of training samples.

Backpropagation Explained

Backpropagation passes error information backward through the network to determine how much each individual weight contributed to the total loss. It uses the mathematical **Chain Rule** from calculus to calculate the partial derivative of the cost function with respect to every single weight (w) and bias (b):

$$ rac{\partial J}{\partial w_{ij}} = rac{\partial J}{\partial \hat{y}} \cdot rac{\partial \hat{y}}{\partial z} \cdot rac{\partial z}{\partial w_{ij}}$$

These partial derivatives, called **gradients**, act like a directional compass: they show how much the total error would change if a specific weight value were increased or decreased slightly.

6. Understanding Gradient Descent Optimization

Once backpropagation calculates the gradients, the network uses an optimization algorithm called **Gradient Descent** to update its parameters. The algorithm adjusts the weights in the opposite direction of the calculated gradient to slide down the error curve toward the lowest possible value (the minimum loss).

$$w_{ ext{new}} = w_{ ext{old}} - lpha rac{\partial J}{\partial w_{ ext{old}}}$$

The parameter $ lpha$ (Alpha) represents the **Learning Rate**. This hyperparameter controls the size of the steps the model takes down the error curve:

- **Too Large:** The updates can overshoot the lowest point completely, causing the model to oscillate and fail to converge.
- **Too Small:** The network takes tiny steps, making training extremely slow and prone to getting stuck in flat areas or local minima.

7. Stochastic Gradient Descent (SGD) vs. Batch Gradient Descent

The way data is fed into the optimization algorithm significantly changes training speed, memory usage, and how smoothly the model converges.

Optimization Mode	Core Operational Methodology	Trade-offs & Performance Dynamics
Batch Gradient Descent	Computes the gradients across the entire training dataset before executing a single parameter update step.	Provides stable, smooth updates directly toward the minimum, but requires significant memory and updates slowly on large datasets.
Stochastic Gradient Descent (SGD)	Updates weights immediately after processing each individual training sample.	Fast and memory-efficient with a noisy path that helps bounce out of poor local minima, though it never completely settles at the exact minimum.
Mini-Batch Gradient Descent	Splits the dataset into small, manageable subsets (typically batches of 32, 64, or 128 samples) for updates.	The standard choice in modern deep learning. Combines the speed of vector processing with the stable convergence of batch updates.